Book a Doctor

Full Stack Development with MERN - Project Documentation

1. Introduction

Project Title: Book a Doctor

Team Members:

Navendhiran C— Full Stack Developer (Frontend, Backend, Database, Deployment) (Add additional team members and their roles here if applicable.)

2. Project Overview

Purpose:

Book a Doctor makes it easier for patients to connect with healthcare providers. Patients can search for, schedule, and manage appointments through a simple interface, while doctors and admins get straightforward, secure tools for handling schedules, verification, and overall platform activity.

Features:

- Role-based registration and login for Patients, Doctors, and Admins (JWT authentication)
- Doctor search and filtering by specialization, location, rating, and availability
- Live appointment booking, along with rescheduling and cancellation
- Secure upload and storage of medical reports and prescriptions
- Admin dashboard for verifying doctors and monitoring the platform
- Email/SMS alerts for booking confirmations and reminders

3. Architecture

Frontend (React):

The frontend is built in React.js with a component-based layout. React Router manages navigation between pages (login, doctor search, booking, dashboards), Axios handles calls to the backend REST API, and Bootstrap keeps the UI responsive. Components are grouped by role (Patient, Doctor, Admin), with shared pieces such as navigation bars and forms reused across roles.

Backend (Node.js & Express.js):

The backend follows the MVC (Model-View-Controller) pattern. Express.js handles routing and middleware (JSON parsing, CORS, JWT authentication). Controllers contain the business logic for each domain (Auth, Doctor, Appointment, Document, Admin), keeping routes thin and logic centralized and testable.

Database (MongoDB):

MongoDB holds three core collections through Mongoose schemas:

- Users — name, email, hashed password, role (patient/doctor/admin), profile info
- Doctors — linked to a User, specialization, experience, availability, verification status
- Appointments — references to Patient and Doctor IDs, date/time slot, status (pending/confirmed/cancelled), uploaded document references

4. Setup Instructions

Prerequisites:

- Node.js (v16 or higher) and npm
- MongoDB (local installation or a MongoDB Atlas cluster)
- Git

Installation:

1. Clone the repository:

```
git clone <repository-url>
cd book-a-doctor
```

Install backend dependencies:

```
cd server
npm install
```

Install frontend dependencies:

```
cd client
npm install
```

Create a .env file inside the server folder with:

```
MONGO_URI=your_mongodb_connection_string JWT_SECRET=your_jwt_secret_key
PORT=5000
```

5. Folder Structure

Client (React frontend):

```
client/src/ components/ (navbar, forms, cards)
pages/ (Login, Register, DoctorList, BookAppointment, Dashboard) services/ (axios API call
functions)
context/ (auth context for logged-in user & token)
App.js, index.js
```

Server (Node.js backend):

```
server/ models/ (User.js, Doctor.js, Appointment.js)
routes/ (authRoutes.js, doctorRoutes.js, appointmentRoutes.js, adminRoutes.js)
controllers/ (authController.js, doctorController.js, appointmentController.js, adminController.js) middleware/ (authMiddleware.js —JWT
verification&role checks)
config/ (db.js —MongoDB connection)
server.js
```

6 Running the Application

Backend:

```
cd server
npm start
```

Frontend:

```
cd client
npm start
```

By default the frontend serves at http://localhost:3000 while the backend API listens on http://localhost:5000.

7. API Documentation

Method	Endpoint	Description	Request Body / Params
POST	/api/auth/register	Register a new user (patient/doctor)	{ name, email, password, role }
POST	/api/auth/login	Authenticate a user and return a JWT token	{ email, password }
GET	/api/doctors	Get list of doctors (supports specialization/location filters)	Query: ?specialization=&l ocation;=
GET	/api/doctors/:id	Get a single doctor's profile	Param: doctor id
PUT	/api/doctors/:id	Update doctor profile / availability (Doctor only)	{ specialization, availability, ... }
POST	/api/appointments	Book a new appointment (Patient only)	{ doctorId, date, timeSlot }
GET	/api/appointments/user/:u serId	Get all appointments for a patient or doctor	Param: user id
PUT	/api/appointments/:id/can cel	Cancel or reschedule an appointment	{ status, newDate? }
POST	/api/documents/upload	Upload a medical document for an appointment	multipart/form-data: file, appointmentId
PUT	/api/admin/doctors/:id/ap prove	Approve a pending doctor registration (Admin only)	Param: doctor id

Example response (POST /api/auth/login):

```
{"token":"eyJhbGciOi...","user":{"id":"64f...","name":"Jane Doe","email":
"jane@example.com","role":"patient"}}
```

8. Authentication

Authentication runs on JSON Web Tokens (JWT). Once login succeeds, the server signs a token that carries the user's ID and role; the client keeps this token (e.g., in memory or React context) and sends it in the Authorization header (Bearer token) on later requests.

- Passwords are hashed with bcrypt before being stored — plain-text passwords are never saved.
- An authentication middleware checks the JWT on protected routes and blocks invalid or expired tokens.
- Role-based authorization limits certain endpoints — for example, only Admins can approve doctors, and only Doctors can edit their own profile.

9. User Interface

[Insert screenshots or GIFs here showcasing the Login/Register page, Doctor Search page, Doctor Profile page, Booking flow, Patient Dashboard, Doctor Dashboard, and Admin Dashboard.]

10. Testing

API endpoints were checked with Postman to confirm request/response handling, status codes, and authentication behavior. Manual User Acceptance Testing (UAT) covered end-to-end flows including registration, doctor search, appointment booking, cancellation, and document upload (refer to the separate UAT document for the detailed test cases and results).

11. Screenshots or Demo

[Add screenshots of key screens here, or a link to a demo video / hosted deployment showcasing the working application, as required by your internship deliverables.]

12. Known Issues

- The doctor's available slots list doesn't update in real time after a patient cancels an appointment (a manual refresh is needed).
- Document upload file size isn't validated on the frontend, so oversized files can trigger a server-side error.
- The admin's doctor-approval button stays clickable for a moment after submission, which could allow a duplicate approval click.

13. Future Enhancements

- Video/audio teleconsultation for remote appointments
- Online payment gateway for consultation fees
- Doctor ratings and patient reviews
- In-app real-time chat between patient and doctor
- Multi-language support for wider accessibility
- Push notifications and calendar sync for appointment reminders